Ders 15 Genişletilmiş Notu: Uygulama 1: Shell Programlamaya Giriş, Dosya İşlemleri, İzinler, Değişkenler ve Echo

Ders 15 Genişletilmiş Notu: Uygulama 1: Shell Programlamaya Giriş, Dosya İşlemleri, İzinler, Değişkenler ve Echo

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulama Oturumu ve Dönem Planı
- 2. Kabuk (Shell) Nedir?
- 3. Temel Dosya ve Klasör İşlemleri
- 4. Kullanıcı ve Grup Yönetimi
- 5. Dosya İzinleri: `ls -l`, `chmod`, `chown`
- 6. İlk Kabuk Betiği: `clear` ve `echo`
- 7. `echo -e` ve Kaçış Dizileri
- 8. Sistem Değişkenleri
- 9. Kullanıcı Değişkenleri ve `$` İşareti
- 10. Aritmetik İşlemler: `expr`
- 11. Kullanıcıdan Veri Alma: `read`
- 12. Joker Karakterler: * ve ?
- 13. Dosyaya Yazma ve Sona Ekleme: > ve >>
- 14. Linux Dışında Kabuk Betiği
- Örnek Sorular
- Dersin Toparlaması

1. Uygulama Oturumu ve Dönem Planı

Bu kayıt, İşletim Sistemleri dersinin ilk uygulama (laboratuvar) oturumudur. Oturum Zoom üzerinden yapıldı. Dersin hocası oturumu açtı, yönetimi uygulamayı yürüten öğretim elemanına devretti ve ayrıldı. Anlatımın tamamını bu öğretim elemanı yaptı. Oturuma dersin üç grubundan da öğrenciler katıldı. Asıl anlatım yaklaşık 00:20'de başlar; öncesi katılım bekleme ve öğrenci sorularıdır.

Öğretim elemanı dönemin uygulama planını şöyle anlattı:

- Kabuk (shell) programlama: iki hafta. Bu hafta temel komutlar ve "know-how" seviyesi. Gelecek hafta iki komutu birleştirerek daha karmaşık işler, bunların betiğe gömülmesi, döngüler, koşullar ve sözdizimi.
- Süreçler arası iletişim (interprocess communication) ve iş parçacıkları (thread): iki hafta daha.
- Çekirdek modülü (kernel module) ile ilgili bir uygulama.
- Toplam beş uygulama olacak.

Uygulamaların nasıl değerlendirileceği henüz netleşmemişti. Uygulamaların hepsini ya da birkaçını içeren bir proje verilebilir. Uygulamalarda anlatılanlar sınav sorusuna da dönüşebilir. İkisi birlikte de olabilir.

> **Hocanın vurgusu:** Uygulamalara katılmanız şiddetle tavsiye ediliyor. Laboratuvar mı, proje mi olacağı belli değil, ama uygulamalardan değerlendirileceksiniz.

Soru: Bugünkü konular dağıtımdan (distribution) bağımsız mı olacak? **Cevap:** Evet. Bütün komutlar BASH üzerinden anlatılacak. BASH hemen her dağıtımda vardır, yoksa da sonradan kurulabilir. Debian ya da Ubuntu kullanmanız fark etmez. Yardımcı (utility) komutlar ortaktır. Kabukların sözdizimi biraz ayrışsa da büyük ölçüde aynıdır.

Öğretim elemanı oturumda kaç kişinin tamamen Linux üzerinde çalıştığını sordu; iki üç kişi el kaldırdı. Bu kişiler anlatılanların çoğuna aşina olacaktı, ama anlatım en baştan yapıldı.

Kabuk ile Python karıştırılmamalıdır. Python daha basit görünebilir, ama kabuk başka amaçlar için kullanılır. Ubuntu kurulduğunda Python gelmeyebilir, ama kabuk her zaman gelir. Hiçbir şey kurmadan doğrudan betik yazmaya başlayabilirsiniz. Bu önemli bir avantajdır.

2. Kabuk (Shell) Nedir?

Shell kelimesi “kabuk” demektir ve bu adın bir nedeni vardır. Bilgisayar yazılımının, yani işletim sisteminin çekirdeğini çekirdek (kernel) oluşturur. Kabuk, bu çekirdeğin üzerine kurulmuş bir programdır. İşletim sisteminin servislerine erişmek için genel olarak kabuk kullanılır.

Çekirdekte dosya yönetimi (file management), süreç yönetimi (process management), aygıt yönetimi (device management) ve G/Ç (I/O) gibi işlemler gerçekleşir. Masaüstünde yaptığınız bütün işlemler de aslında bu tür işlemlerdir. Kabuk dilleri bu işlemleri açıkça yapmayı sağlar. Kullanımları pratiktir, betikler (script) şeklinde yazılır. Çekirdeğe yakın olmaları da bir avantajdır. Bu yüzden birçok kişi basit işler için kabuk programlarını tercih eder.

Farklı işletim sistemleri için birden çok kabuk vardır:

- BASH (Bourne Again Shell): bu uygulamada kullanılan kabuk.
- C kabuğu (C shell, CSH).
- Korn kabuğu (Korn shell, KSH).
- PowerShell: Windows için çıkarılmış, son yıllarda yaygınlaşmış kabuk.

Uygulama Ubuntu üzerinde, Ubuntu’da kurulu gelen BASH ile anlatıldı. Öğretim elemanı, ilk gösterilen komutların çoğunun aslında kabuğun kendi komutu değil, Linux’un yardımcı (utility) programları olduğunu belirtti.

3. Temel Dosya ve Klasör İşlemleri

3.1 Listeleme: `ls`

`ls` komutu bulunulan klasördeki (dizin, directory) dosyaları listeler. Öğretim elemanı masaüstündeki bir klasörde çalışıyordu. Windows’ta bunun karşılığı `dir` komutudur. Öğretim elemanı `dir`’in Ubuntu’da da çalıştığını görünce bunun bir takma ad (alias) olarak tanımlandığını söyledi.

> **Ders sonrası not:** Derste Ubuntu’da `dir`’in bir takma ad olduğu söylendi; doğrusu, `dir` GNU coreutils içinde ayrı bir komuttur. `ls`’ye benzer bir çıktı verir, ama takma ad değildir.

Kullanılan terminal dosya türlerini renkle gösteriyordu: klasörler mavi, sıradan dosyalar beyaz.

3.2 Klasör ve Dosya Oluşturma: `mkdir`, `touch`

Klasör oluşturmak için `mkdir` (make directory) yazılır, ardından klasörün adı verilir. Boş bir metin dosyası oluşturmak için `touch` kullanılır:

```
mkdir klasor
touch dosya
ls
```

3.3 Klasörler Arasında Gezinme: `cd`, `.` ve `..`

Bir klasöre geçmek için `cd` (change directory) yazılır. Derste `cd klasor` ile içeri girildi, `touch` ile bir dosya oluşturuldu, sonra bir üst klasöre çıkıldı. İki nokta (`..`) bir üst klasör, tek nokta (`.`) bulunulan klasör demektir.

> **Ders sonrası not:** Derste tek nokta için "şu anki klasördeki bütün dosyalar, any demek" dendi; doğrusu, `.` yalnızca bulunulan klasörün kendisidir. "Bütün dosyalar" anlamı * joker karakterine aittir (Bölüm 12).

Ekranı temizlemek için `clear` kullanılır.

3.4 Silme: `rm`, `rmdir`, `rm -r`

`rm` (remove) dosya siler. `rmdir` (remove directory) klasör siler, ama yalnızca **boş** klasörü. Derste bunu göstermek için klasörün içine `dosya2` adında bir dosya kondu ve `rmdir klasor` çalıştırıldı. Komut beklendiği gibi hata verdi ("Directory not empty"). Sonra içteki dosya `rm` ile silindi ve `rmdir` bu kez başarılı oldu.

Dosyayı silmek için klasörün içine girmek gerekmez; `rm klasor/dosya2` gibi yol verilebilir. Tab tuşu dosya ve klasör adını tamamlar: Adın ilk harflerini yazıp Tab'a basınca, o harflerle başlayan tek dosya varsa adı tamamlanır.

```
mkdir klasor
touch klasor/dosya2
rmdir klasor      # hata: klasör boş değil
rm klasor/dosya2
rmdir klasor      # şimdi başarılı
```

3.5 İç İçe Klasörler ve `rm -r`

Sohbette bir öğrenci, içinde dosya olan bir klasörün tek seferde silinip silinemeyeceğini sordu. Öğretim elemanı bunun için özyinelemeli (recursive) silme gerektiğini söyledi ve gösterdi. Önce iç içe klasörler oluşturuldu: `mkdir klas1`, ardından `mkdir klas1/klas2`. Sonra `rm -r klas1` ile hepsi silindi.

```
mkdir klas1
mkdir klas1/klas2
rm -r klas1       # klas1 ve içindeki her şey silinir
```

`rm -r` bir klasörü verdiğinizde, içindeki bütün alt klasörleri ve dosyaları özyinelemeli olarak siler.

Soru: Önce `klas1`'i oluşturmadan doğrudan `klas1/klas2` oluşturulabilir mi? **Cevap:** Öğretim elemanı denedi; düz `mkdir klas1/klas2` hata verdi, çünkü `klas1` yoktu. Bir öğrenci sohbette `-p` seçeneğini önerdi. Denendi ve çalıştı: `mkdir -p klas1/klas2` eksik üst klasörleri de oluşturur. Bu işlemler `sudo` olmadan, normal kullanıcı yetkisiyle yapıldı.

3.6 Taşıma: `mv`

Yeni bir klasör (`klasor_yeni`) oluşturuldu, içine `dosya1` kondu. Sonra dosya bir üst klasöre taşındı:

```
mv dosya1 ..
```

`mv`'ye önce taşınacak dosya, sonra yeni konumu verilir. Taşımadan sonra klasörde dosya kalmadı, üst klasörde göründü.

3.7 Dosya İçeriği: `cat`, `gedit`, `nano`

`cat dosya1` dosyanın içeriğini ekrana yazar. Dosya boşken ekrana hiçbir şey gelmedi. Dosyaya bir şey yazmak için iki editör gösterildi:

- `gedit`: Ubuntu'daki grafik arayüzlü metin editörü. `gedit dosya1` ile açıldı, bir şeyler yazıldı, kaydedilip kapatıldı. Sonra `cat dosya1` yazılan içeriği gösterdi.

- `nano`: terminal içinde çalışan editör. Ayrı bir pencere açmadan dosya düzenlenir. Kaydedip çıkmak için Ctrl+X kullanıldı.

4. Kullanıcı ve Grup Yönetimi

Her dosyayı her kullanıcı silememeli. Bir işletim sistemini birden çok kullanıcı kullanıyorsa, yetkilerinin tanımlı olması gerekir. Kullanıcı ekleyip silmek için yönetici yetkisi gerekir. Bir komutu yönetici (root) yetkisiyle çalıştırmak için başına `sudo` yazılır.

Slayttaki komutlar:

- `sudo useradd ...`: kullanıcı ekler.
- `sudo usermod ...`: kullanıcının özelliklerini değiştirir, yetkilendirir.
- `sudo userdel ...`: kullanıcı siler.
- `sudo groupadd grup`: grup oluşturur.
- `sudo usermod -aG grup kullanici`: kullanıcıyı gruba ekler.
- `sudo groupdel grup`: grubu siler.

Derste `test user` adlı bir kullanıcı oluşturulmaya çalışıldı. Komut seçenekleri ilk denemede hata verdi. Daha sonra var olmayan bir kullanıcıyı silmeye çalışınca "kullanıcı yok" hatası, önceki deneme kullanıcısını silmeye çalışınca "cannot lock" ve "not found" hataları geldi. Öğretim elemanı kullanıcının tam oluşturulamadığını düşündü ve bu kısmı atladı.

Kullanıcıları listeleyen dosyanın adı derste hatırlanamadı.

> **Ders sonrası not:** Sistemdeki kullanıcılar `/etc/passwd` dosyasında tutulur; `cat /etc/passwd` ile listelenebilir. Parola özetleri ayrı olarak `/etc/shadow` dosyasındadır.

Bir grubun var olup olmadığını görmek için:

```
cat /etc/group | grep grup_adi
```

Burada `cat` grup dosyasını yazdırır. `grep`, önceki komutun çıktısı üzerinde arama yapar. İki komut boru (pipe, `|`) işaretiyle bağlanır. Slaytta boru işaretinin yerine yanlışlıkla `/` yazılmıştı; derste düzeltildi. Komutları boruyla birleştirme gelecek hafta ayrıntılı anlatılacak.

> **Hocanın vurgusu:** Kullanıcı ve grup ekleme-silme komutları o kadar önemli değil, büyük olasılıkla çok yazmayacaksınız. Ama `chmod` ve `chown` komutlarını çok sık kullanacaksınız.

5. Dosya İzinleri: `ls -l`, `chmod`, `chown`

Erişim yetkiniz olmayan dosyalarla işlem yapmaya çalıştığınızda sürekli "permission denied" hatası alırsınız. Bunu aşmak için dosyaya uygun izni vermek gerekir.

5.1 `ls -l` Çıktısını Okumak

Derste `folder` adında bir klasör ve içinde `file` adında bir dosya oluşturuldu. `ls -l` ile dosyanın ayrıntıları gösterildi: oluşturulma saati, dosyanın sahibi (kullanıcı) ve grubu, izinleri. İzin dizisi şöyleydi:

```
-rw-r--r--
```

Her `-` işareti, henüz verilmemiş bir izin demektir. `r` okuma (read), `w` yazma (write), `x` çalıştırma (execute) iznidir.

Bu dizinin yapısı:

- İlk karakter: dosya tipi.
- Sonraki üç karakter: dosyanın sahibi (user) için izinler.
- Sonraki üç karakter: dosyanın grubu (group) için izinler.
- Son üç karakter: geri kalan herkes (others) için izinler.

Dosya tipi karakterleri öğrencilerle birlikte sayıldı:

- `-`: sıradan dosya.
- `d`: klasör (directory).

- `l`: sembolik bağ (symbolic link).
- `c`: karakter aygıtı (character device).
- `b`: blok aygıtı (block device).
- `p`: adlandırılmış boru (named pipe).
- `s`: soket (socket).

Öğretim elemanı bunların çoğunu pek görmediğini söyledi, ama bir örnek gösterdi: Sanal makinede macOS ile ortak, eşzamanlanan bir paylaşım klasörü vardı. `ls -l` çıktısında bu klasörün başında `l` vardı, yani sembolik bağdı. Terminal de onu farklı bir renkle gösteriyordu.

5.2 `chmod` ile İzin Vermek

İzinler `chmod` (change mode) ile verilir. İzinler sayıyla yazılır. Her basamak bir kullanıcı sınıfına (sahip, grup, diğerleri) karşılık gelir. Her basamak 3 bittir: okuma 4, yazma 2, çalıştırma 1 değerindedir (ara adım eklendi). Örnekler:

- `chmod 777 file`: herkese okuma, yazma ve çalıştırma izni. Sonuç: `-rwxrwxrwx`.
- `chmod 111 file`: herkese yalnızca çalıştırma izni. Sonuç: `---x--x--x`.
- `chmod 000 file`: bütün izinleri kaldırır. Sonuç: `----------`.

Bir basamak neden en fazla 7 olabilir? Üç izin var: okuma, yazma, çalıştırma. Her biri verilir ya da verilmez, yani 1 bittir. Üç bit $2^3 = 8$ farklı kombinasyon verir. Bunlar 0'dan 7'ye kadar rakamlarla gösterilir. Bu yüzden `chmod 99 file` geçersizdir ve hata verir.

$$7 = 4 + 2 + 1 = r + w + x, \qquad 6 = 4 + 2 = r + w, \qquad 5 = 4 + 1 = r + x$$

5.3 `chmod 000` Denemesi

`chmod 000 file` sonrasında `cat file` çalıştırıldı ve "Permission denied" hatası geldi. Dosyanın sahibi bile artık dosyayı okuyamıyordu. Okuyamadığınız bir dosyaya yazamazsınız da. `chmod 777 file` ile izinler geri verildi.

Bu sırada `nano file` ile dosya açılmaya çalışıldı ve nano bir ekran açtı. Öğretim elemanı bunu "nano büyük olasılıkla otomatik root yetkisiyle açılıyor" diye açıkladı.

> **Ders sonrası not:** Öğretim elemanı nano'nun root yetkisiyle açıldığını söyledi; doğrusu, `nano` onu çalıştıran kullanıcının yetkisiyle çalışır. İzni olmayan dosyayı okuyamaz; boş bir tampon açar ve alt satırda izin hatası gösterir. Kaydetmeye çalışınca da yazma hatası verir.
>
> **Hocanın vurgusu:** Bu kısım önemli. İzin vermezseniz dosyayı yazmayı, çalıştırmayı geçtim, okuyamıyorsunuz bile.

`chown` (change owner) ise dosyanın sahibini değiştirir. Derste örnek olarak çalıştırılmadı; yalnız `chmod` ile birlikte çok kullanılacağı söylendi.

> **Ek bilgi (derste anlatılmadı):** `chown` genelde `sudo chown kullanici:grup dosya` biçiminde kullanılır. Sayısal yazım yerine `chmod u+x dosya` gibi harfli yazım da vardır: `u` sahip, `g` grup, `o` diğerleri, `+` izin ekler, `-` kaldırır.

6. İlk Kabuk Betiği: `clear` ve `echo`

Dosya ve kullanıcı işlemlerinden sonra kabuk programlamaya geçildi. Öğretim elemanı her slayt için masaüstünde hazır bir betik dosyası oluşturmuştu. İlk betik, her dilde olduğu gibi ekrana yazı yazdırıyordu. `cat` ile içeriği gösterildi:

```
clear
echo "Hello World"
```

`clear`, terminalde komut olarak yazılan `clear` ile aynıdır: ekranı temizler. Ardından `echo` ekrana yazıyı yazar. Bir betiği çalıştırmak için `./` yazıp dosya adını vermek yeterlidir:

```
./01.sh
```

C'deki ya da Python'daki `print`, kabukta `echo`'ya karşılık gelir. `echo` yazdıktan sonra yeni satıra geçer, yani `print`'ten çok `println` gibi davranır.

> **Ek bilgi (derste anlatılmadı):** `./dosya` ile çalıştırmak için dosyanın çalıştırma (`x`) izni olmalıdır: `chmod +x dosya.sh`. Betiğin ilk satırına genelde `#!/bin/bash` yazılır; bu satır, betiği hangi kabuğun çalıştıracağını belirtir. İzin yoksa betik `bash dosya.sh` ile de çalıştırılabilir.

7. `echo -e` ve Kaçış Dizileri

7.1 Sistem Bilgisi Yazdıran Betik

İkinci betik birkaç sistem bilgisini yazdırıyordu:

- `echo "... $USER"`: o anki kullanıcının adı. `$USER` bir sistem değişkenidir.
- `date`: bugünün tarihi.
- `who`: oturum açmış kullanıcılar.
- `cal`: takvim.

`$USER`'ın sistem değişkeni olduğunu nereden anlarız? Büyük harfle yazılmıştır. Sistem değişkenlerini büyük harfle yazmak yaygın bir uygulamadır. Kendi değişkenlerinizi küçük harfle yazarsanız sistem değişkenleriyle karışmaz.

Betikte tarih satırında yazının sonunda `\c` kaçış dizisi vardı. Öğretim elemanı slayttaki betiği değiştirmiş, iki `echo` satırından yalnız birine `-e` eklemişti. Çalıştırınca fark görüldü: `-e` olan satırda `\c` işlendi; `-e` olmayan satırda `\c` ekrana olduğu gibi yazıldı.

Ders sonrası denendiğinde aynı sonuç alınır:

```
echo -e "Tarih: \c"; echo "SON"     # çıktı: Tarih: SON
echo "Tarih: \c"                    # çıktı: Tarih: \c
```

> **Hocanın vurgusu:** Kaçış dizilerinin çalışması için `echo` yeterli değil, genişletilmiş (extended) ifadeleri açan `-e` parametresini eklemeniz gerekiyor. Slaytta bu yazmıyor, ama doğrusu bu.
>
> **Ders sonrası not:** Derste `\c` için "clear demek" dendi; doğrusu, `echo -e` içinde `\c` kendisinden sonraki çıktıyı keser ve sona yeni satır eklemez. Bu yüzden yukarıdaki örnekte "SON" aynı satıra yazıldı. Ekranı temizleyen şey `clear` komutudur.

Soru: Takvim neden çalışmadı? **Cevap:** Bir öğrencinin sorusu üzerine betik açıldı. Takvim komutunun bulunduğu satırda iki komutu ayıran noktalı virgül eksikti. Noktalı virgül eklenince takvim geldi. Öğretim elemanı, slaytların ve kodların güncel olmadığını, noktalı virgül gibi küçük sözdizimi kaymaları olabileceğini söyledi.

7.2 Kaçış Dizileri Betiği

Üçüncü betik `echo -e` ile kullanılan kaçış dizilerini tek tek gösteriyordu:

Dizi	Etkisi
`\a`	Uyarı (alert): işletim sisteminin uyarı sesini çıkarır
`\b`	Geri silme (backspace): önceki karakteri siler
`\c`	Sonraki çıktıyı keser, sona yeni satır koymaz
`\n`	Yeni satır (new line)
`\r`	Satır başı (carriage return)
`\t`	Sekme (tab)
`\\`	Ters eğik çizginin kendisi

Ters eğik çizgi özel karakterleri belirtmek için kullanıldığından, metinde ters eğik çizginin kendisini yazmak gerekirse o da kaçış karakteriyle yazılır: `\\`.

Çalıştırınca: ilk satırda yazı yazıldı ve aynı anda uyarı sesi geldi. İkinci satırda bir önceki karakter silindi. Sonra yeni satıra geçildi ve fazladan bir boşluk satırı görüldü. Ters eğik çizginin kaçışla yazıldığı görüldü. Sekme de vardı, ama ekranda boşluk olarak görünüyordu.

Ders sonrası denemede `echo -e "a\tb|c\bD|x\\\\y"` komutu `a`, sekme, `b|`, sonra `c`'nin üzerine `D` ve `|x\y` yazdı.

> **Hocanın vurgusu:** Bunların hepsini çalıştırmak için `echo`'nun yanına `-e` yazmanız gerekiyor. Yoksa bu dizilerin hepsini görmezden geliyor.

8. Sistem Değişkenleri

Kabukta iki tür değişken vardır:

- **Sistem değişkenleri:** sistemle birlikte gelen değişkenler.
- **Kullanıcı değişkenleri:** kendi tanımladığımız değişkenler.

Sistem değişkenlerini genel kültür olarak bile bilmek gerekir, çünkü çok kullanılırlar. Windows'taki karşılığı ortam değişkenleridir (environment variables). Bazı programları kurarken bunlarla karşılaşmışsınızdır. Örneğin Java çalışma ortamını kurduktan sonra Eclipse'in hangi Java ile çalışacağını bu değişkenlerdeki yollar (path) belirler.

Sistem değişkenleri kurulumlarda ve iki programın ya da iki servisin birbiriyle haberleşmesinde ortak kullanılan değerleri tanımlar. Bu değişkenlere parametre ekleyebilir, onları genişletebilir ya da yeni değişkenler oluşturabilirsiniz.

8.1 `set` ile Listeleme

Bütün değişkenleri görmek için `set` komutu kullanıldı. Çıktının bir kısmı fonksiyonlardı; listenin başına gidince daha anlamlı değişkenler görüldü: makine tipi, makine adı, yapılandırmalar. Bir değişken `HTTP_PROXY` idi ve 8080 portunu gösteriyordu. Öğretim elemanı bunu göstermek için sanal makinede açık olan Burp Suite adlı vekil sunucu (proxy) aracını gösterdi. Bu araç 8080 portunda çalışıyordu ve bazı uygulamalar HTTP isteklerini bu değişken üzerinden onun aracılığıyla yapıyordu. Bu değişken bir kullanıcı değişkeni değildi; bir program tarafından sisteme eklenmişti ve büyük harfle yazılmıştı.

> **Ders sonrası not:** `set` yalnız ortam değişkenlerini değil, kabuğun bütün değişkenlerini ve fonksiyonlarını listeler; çıktıda fonksiyonların görülmesinin nedeni budur. Yalnızca ortam değişkenlerini görmek için `env` ya da `printenv` kullanılır.

8.2 Bazı Sistem Değişkenleri

`set` çıktısı süzülerek bazı değişkenler gösterildi:

- `COLUMNS`: terminalin kaç sütun genişliğinde olduğu. Derste 94'tü.
- `LINES`: terminalin kaç satır olduğu. Derste 28'di.
- `HOME`: kullanıcının ev klasörü. Süzme yapınca `HOME` geçen başka değişkenler de geldi.
- `PWD`: o an bulunulan klasör.
- `USER`: o anki kullanıcı adı.
- `SHELL`: kullanılan kabuk.

> **Hocanın vurgusu:** Kendi değişkeninizi oluştururken küçük harf kullanmaya dikkat edin ki karışmasın. Çünkü bu sisteme erişen tek kullanıcı siz değilsiniz.

9. Kullanıcı Değişkenleri ve `$` İşareti

Sıradaki betik değişken tanımlamayı gösteriyordu. Değişken adları, birçok programlama dilinde olduğu gibi, büyük-küçük harfe duyarlıdır: Biri büyük, biri küçük harfle yazılmış aynı adlı iki değişken farklı değişkenlerdir.

```
degisken0=
degisken1=10
degisken2="deneme"
degisken3=deneme
degisken4=`pwd`
echo $degisken0 $degisken1 $degisken2 $degisken3 $degisken4
```

Çalıştırınca sonuç şöyle yorumlandı:

- `degisken0` boştu, ekrana bir şey yazılmadı.
- `degisken1` 10'du, 10 yazıldı.
- `degisken2` ve `degisken3` ikisi de "deneme" yazdırdı. Birinde tırnak var, birinde yok, ama ikisi de kabul edildi ve hata alınmadı.
- `degisken4`'e `pwd` komutunun sonucu atanmıştı. `pwd` o anki klasörü döndürür. Bu değer başka bir değişkene aktarılıp `echo` ile yazdırılabilir.

Toparlarsak: Bir değişkeni tanımlarken başına `$` koymayız, `=` ile atama yaparız. Değişkenin içindeki değeri kullanırken ya da yazdırırken başına `$` koyarız. `PWD` gibi sistem değişkenlerinin değerine erişmek için de başına `$` yazmak gerekir.

Öğretim elemanı bunu C ile karşılaştırdı: C'de değişkeni tanımlarken tipini (int, char) yazarsınız; yazdırırken `%d` ya da `%f` gibi bir biçim belirtirsiniz. Kabuktaki `$` de değişkenin içindeki değere erişmeyi sağlayan işarettir.

> **Ders sonrası not:** Derste `$` C'deki adres ve `%d` biçim belirteciyle benzetildi; doğrusu, `$` bir adres ya da biçim belirteci değildir. Kabuk, komutu çalıştırmadan önce `$ad` yazan yeri değişkenin değeriyle değiştirir. Buna parametre genişletmesi (parameter expansion) denir. Benzetme yalnızca anlaşılır olsun diye yapıldı.
>
> **Hocanın vurgusu:** Bu çok önemli bir ayrıntı. Başlarken çok unutulabiliyor, dikkat etmeniz gerekiyor.
>
> **Ek bilgi (derste anlatılmadı):** Atamada `=` işaretinin çevresinde boşluk olmamalıdır. `x = 5` yazılırsa kabuk `x`'i bir komut sanar ve "command not found" hatası verir. Ters tırnak yerine bugün çoğunlukla aynı işi yapan `$(pwd)` biçimi kullanılır.

10. Aritmetik İşlemler: `expr`

Aritmetik işlemler için `expr` (expression) kullanılır. `expr 1 + 2` yazdığınızda bunun matematiksel bir ifade olduğunu belirtirsiniz, sistem işlemi yapar. Toplama, çıkarma, çarpma, mod alma gibi işleçler vardır; bunlarla da sınırlı değildir.

Betikteki ifadeler ve sonuçları (python ve bash ile doğrulandı):

İfade	Sonuç
`expr 1 + 2`	3
`expr 3 \* 4`	12
`expr 13 % 3`	1
`expr 3 + 4`	7

13 mod 3 için: $13 = 4 \cdot 3 + 1$, kalan 1'dir (ara adım eklendi).

Çarpmada yıldızın önünde ters eğik çizgi vardır. Nedeni derste öğrencilere soruldu (bkz. Örnek Sorular). * kabukta joker karakterdir (wildcard). Kaçış karakteri yazılmazsa kabuk yıldızı klasördeki dosya adlarıyla genişletir ve `expr` yanlış argümanlar alır. Toplamanın kabukta başka bir anlamı olmadığı için +'nın kaçışa ihtiyacı yoktur.

Bir ifadenin sonucunu `echo` içinde yazdırmak için ters tırnak (backtick) kullanılır:

```
echo "Sonuc: `expr 3 + 4`"     # Sonuc: 7
```

Ters tırnak klavyede genelde tilde (~) işaretinin yanındadır; klavye düzenine göre yeri değişir.

Soru: (Ders sonunda bir öğrencinin sohbette sorduğu, yıldızın "pointer gibi algılanmasın diye" yazılıp yazılmadığı sorusu.) **Cevap:** Öğretim elemanı sorunun büyük olasılıkla çarpmadaki ters eğik çizgi ile ilgili olduğunu düşündü. Kabukta yıldız işaretçi (pointer) değil, joker karakterdir; kaçış bu yüzden yazılır.

11. Kullanıcıdan Veri Alma: `read`

Programlar kullanıcıyla etkileşimli olmalıdır. Sıradaki betikte önce "kullanıcı adınız" gibi bir yazı `echo` ile basılıyor, sonra `read` ile değer okunuyordu:

```
echo "Kullanici adiniz:"
read kullanici_adi
echo $kullanici_adi
```

C'deki `printf` kabuktaki `echo`'ya, C'deki `scanf` ise `read`'e karşılık gelir. `read`, kullanıcının yazdığı değeri verilen değişkene koyar. Sonra `$` ile bu değişkenin değeri ekrana yazılır.

12. Joker Karakterler: * ve ?

12.1 Yıldız: *

Yıldızın neden ayrılmış bir karakter olduğu terminalde gösterildi. Kabuk betiğinde yazılan her ifade aslında bir kabuk komutudur, bu yüzden terminalde de doğrudan çalıştırılabilir. Önce `.sh` ile biten dosyalar yıldızla listelenmeye çalışıldı. Bir öğrenci `ls *.sh` yazmayı önerdi; öğretim elemanı bunun da olacağını onayladı. Sonra `.txt` ile biten dosyalar istendi; klasörde tek bir metin dosyası vardı ve o geldi.

Yıldız "herhangi" demektir: Sıfır ya da daha çok sayıda herhangi karakterle eşleşir. `*.txt`, `.txt` ile biten bütün dosyaları döndürür. Öğretim elemanı bunu düzenli ifadelerdeki (regular expression) ve otomatlardaki yıldızla aynı fikir olarak andı. Bu ayrılmış bir karakter olduğu için matematiksel ifadelerde kaçış karakteriyle yazılması gerekir.

Ardından `touch` ile birkaç dosya oluşturuldu (`asd.py` ve adı rakamla başlayan dosyalar):

```
ls *.py      # .py ile biten dosyalar
ls 1*        # 1 ile başlayan bütün dosyalar
```

12.2 Soru İşareti: ?

Soru işareti tam olarak **bir** herhangi karakterle eşleşir. Göstermek için tek karakterlik adı olan bir dosya oluşturuldu (`touch a`), sonra `ls ?` ile listelendi. Slayttaki `ls deneme?` örneği, "deneme" ile başlayıp ardından tek bir karakter gelen dosyaları gösterir.

Ders sonrası denendiğinde `ls deneme?` komutu `deneme1` ve `deneme2`'yi gösterdi, `deneme12`'yi göstermedi.

Derste bir örnekte de soru işaretinin sonda bir karakter gerektirdiği vurgulandı: Uzantının son harfi yerine `?` yazılırsa yalnız tek karakterle tamamlanan adlar eşleşir.

Öğretim elemanı yıldızı çok kullanacağınızı, soru işaretini ise kendisinin pek kullanmadığını söyledi. Bunlara arama süzgeci (search filter) denir; yıldız karakteri joker karakter araması (wildcard search) olarak da anılır.

> **Ders sonrası not:** Derste kabuktaki `?` ile düzenli ifadedeki `?` aynıymış gibi anlatıldı; doğrusu, kabukta `?` tam bir karakterdir. Düzenli ifadede `?` "önceki öğe sıfır ya da bir kez" demektir; herhangi bir tek karakter ise `.` ile gösterilir. Benzer biçimde düzenli ifadede `*` tek başına değil, önceki öğeyle birlikte anlam taşır (`.*` "herhangi karakterler").

13. Dosyaya Yazma ve Sona Ekleme: > ve >>

Son betik dosya işlemlerini gösteriyordu. Betik "deneme" adında bir klasör oluşturmaya çalışıyor, bir dosyayı taşıyor, hata mesajı veriyor, sonra dosyalara `echo` ile bir şeyler yazıp içeriklerini gösteriyordu. İlk çalıştırmada klasör zaten var olduğu için "oluşturulamadı" hatası, `deneme` adlı dosya olmadığı için de "No such file or directory" hatası geldi. Öğretim elemanı önceki yıldan kalan klasörleri temizleyip yeni bir klasörde betiği tekrar çalıştırdı.

Asıl önemli kısım şu dört satırdı:

```
echo 1 > deneme2
echo 3 > deneme2
echo 1 >> deneme2
echo 5 >> deneme2
cat deneme2
```

Dört yazma işlemi yapıldı, beklenen belki `1 3 1 5` idi. Ama `cat deneme2` yalnız üç satır gösterdi: 3, 1, 5. Neden? (bkz. Örnek Sorular)

- Tek `>` yazma (write) demektir. Dosyanın eski içeriğini siler ve en baştan yazar. İmleç dosyanın başına döner.
- Çift `>>` sona ekleme (append) demektir. Bıraktığınız yerden yazmaya devam eder.

Adım adım (bash ile doğrulandı):

1. `echo 1 > deneme2`: dosya `1` olur.
2. `echo 3 > deneme2`: eski içerik silinir, dosya `3` olur.
3. `echo 1 >> deneme2`: sona eklenir, dosya `3 1` olur.
4. `echo 5 >> deneme2`: sona eklenir, dosya `3 1 5` olur.

Bir öğrenci bunları "overwrite" ve "append" olarak adlandırdı. Öğretim elemanı adlarını "write" ve "append" olarak düzeltti: Write zaten dosyaya baştan yazmak demektir.

Betiğin geri kalan kısmı bir yerde sözdizimi hatası nedeniyle beklendiği gibi çalışmadı. Öğretim elemanı burada yalnızca write ile append farkını göstermek istediğini, onun da anlaşıldığını söyledi.

14. Linux Dışında Kabuk Betiği

Komutları boruyla (pipe) birleştirme kısmına zaman yetmedi; bir sonraki uygulamaya bırakıldı.

Soru: Bu betikler yalnız Linux'a mı özgü, Windows'ta da yazılabilir mi? **Cevap:** Çalıştırmak için Linux ortamı gerekir. Windows'ta bunun için Linux alt sistemleri kullanılabilir. Ya da bir sanal makine (VM) kurulur. Yer azsa arayüzsüz bir Debian kurulabilir; yer varsa Ubuntu gibi arayüzlü bir dağıtım kullanılabilir. Windows'un kendi kabuk programları da vardır, örneğin PowerShell; benzer işleri yapar, ama sözdizimi buradakiyle aynı değildir ve hazır gelmeyebilir.

> **Ek bilgi (derste anlatılmadı):** Windows'taki Linux alt sistemi WSL (Windows Subsystem for Linux) adıyla bilinir. Bu alt sistem içinde Ubuntu gibi bir dağıtım kurulur ve BASH betikleri doğrudan çalıştırılabilir.

Örnek Sorular

Bu derste kare ya da video yoktur. Aşağıdaki sorular öğretim elemanının derste sözlü sorduğu ya da öğrencilerin sorduğu sorulardır.

Soru 1 (derste sözlü, 00:38:13). `ls -l` çıktısındaki izin dizisinde `r` okuma, `w` yazma demek. Üçüncü harf `x` neyi gösterir?

> **Çözüm (derste verildi):** Çalıştırma (execute) iznini.

Soru 2 (derste sözlü, 00:38:36). `chmod 777` sonrası izin dizisinde üç tane `rwx` var. Ben tek bir kullanıcıyım; neden bir değil de üç tane var?

> **Çözüm (derste verildi):** Birincisi dosyanın sahibi (user), ikincisi dosyanın grubu (group), üçüncüsü geri kalan herkes (others) içindir.

Soru 3 (derste sözlü, 00:39:03). İzin dizisinin en başındaki karakter ne gösterir?

> **Çözüm (derste verildi):** Dosyanın tipini: `-` sıradan dosya, `d` klasör, `l` sembolik bağ, `c` karakter aygıtı, `b` blok aygıtı, `p` adlandırılmış boru, `s` soket.

Soru 4 (derste sözlü, 00:39:55). `chmod 99 dosya` yazılabilir mi? Bir basamak neden en fazla 7 olabilir?

> **Çözüm (derste verildi):** Yazılamaz; komut hata verir. Her basamak okuma, yazma, çalıştırma için 3 bittir. $2^3 = 8$ kombinasyon vardır ve bunlar 0-7 rakamlarıyla gösterilir. `chmod 000` yazılabilir ve bütün izinleri kaldırır.

Soru 5 (derste sözlü, 00:30:53). Önce üst klasörü oluşturmadan `klas1/klas2` gibi iç içe klasör tek komutla oluşturulabilir mi?

> **Çözüm (derste verildi):** Düz `mkdir klas1/klas2` hata verir, çünkü `klas1` yoktur. `mkdir -p klas1/klas2` eksik üst klasörleri de oluşturur.

Soru 6 (derste sözlü, 00:59:07). `expr` ile çarpma yaparken `3 \* 4` yazılıyor. Toplamada ters eğik çizgi yokken çarpmada neden var?

> **Çözüm (derste verildi):** * kabukta ayrılmış bir joker karakterdir; kaçış yazılmazsa kabuk onu klasördeki dosya adlarıyla genişletir. +'nın kabukta başka bir anlamı olmadığı için kaçışa gerek yoktur. Bu soruya bir öğrenci önce "yorum satırı yapmış" diye cevap verdi; doğru cevap başka bir öğrenciden geldi.

Soru 7 (derste sözlü, 01:08:32). Aşağıdaki dört satırdan sonra dosyada `1 3 1 5` beklenirken neden `3 1 5` var?

```
echo 1 > deneme2
echo 3 > deneme2
echo 1 >> deneme2
echo 5 >> deneme2
```

> **Çözüm (derste verildi):** Tek > yazmadır (write): dosyayı silip baştan yazar, bu yüzden ikinci satır ilk 1'i siler. Çift >> sona eklemedir (append). Sonuç 3, 1, 5 olur (bash ile doğrulandı).

Dersin Toparlaması

Bu ilk uygulama oturumunda kabuğun çekirdek ile kullanıcı arasındaki yeri anlatıldı ve BASH üzerinde temel işlemler yapıldı. Önce dosya ve klasör komutları (`ls`, `mkdir`, `mkdir -p`, `touch`, `cd`, `rm`, `rmdir`, `rm -r`, `mv`, `cat`, `nano`, `gedit`) denendi. Ardından `sudo` ile kullanıcı ve grup yönetimi komutları kısaca geçildi. En çok kullanılacak kısım olarak `ls -l` çıktısı, dosya tipi karakteri, `rwx` izinleri ve `chmod` ile sayısal izin verme vurgulandı. Kabuk programlamada `echo` ve `echo -e` ile kaçış dizileri, sistem ve kullanıcı değişkenleri, `$` ile değere erişme, `expr` ile aritmetik, `read` ile veri alma, * ve ? joker karakterleri ve > ile >> farkı gösterildi.

Öğretim elemanı, gelecek hafta bu haftaki komutları borularla birleştirerek daha karmaşık işler yapmayı, `if` koşullarını, `while`, `until` gibi döngüleri ve bir dosyada sütun seçip veri almak için `awk` komutunu anlatacağını, böylece kabuk programlama kısmını bitireceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.